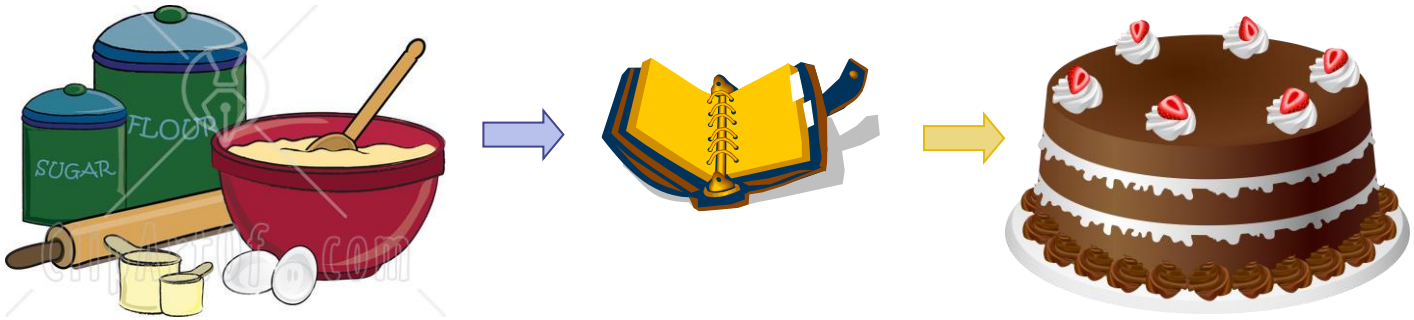
CS 1312
Algorithms Design and Analysis

Introduction

What is an Algorithm



Input **Algorithm** **Output**

- An **algorithm** is a step-by-step procedure for solving a problem.
- The **algorithm** is not an answer for a question, it is a specific set of instructions to get the answer.

What is an Algorithm

- It is any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.
- **Example** :

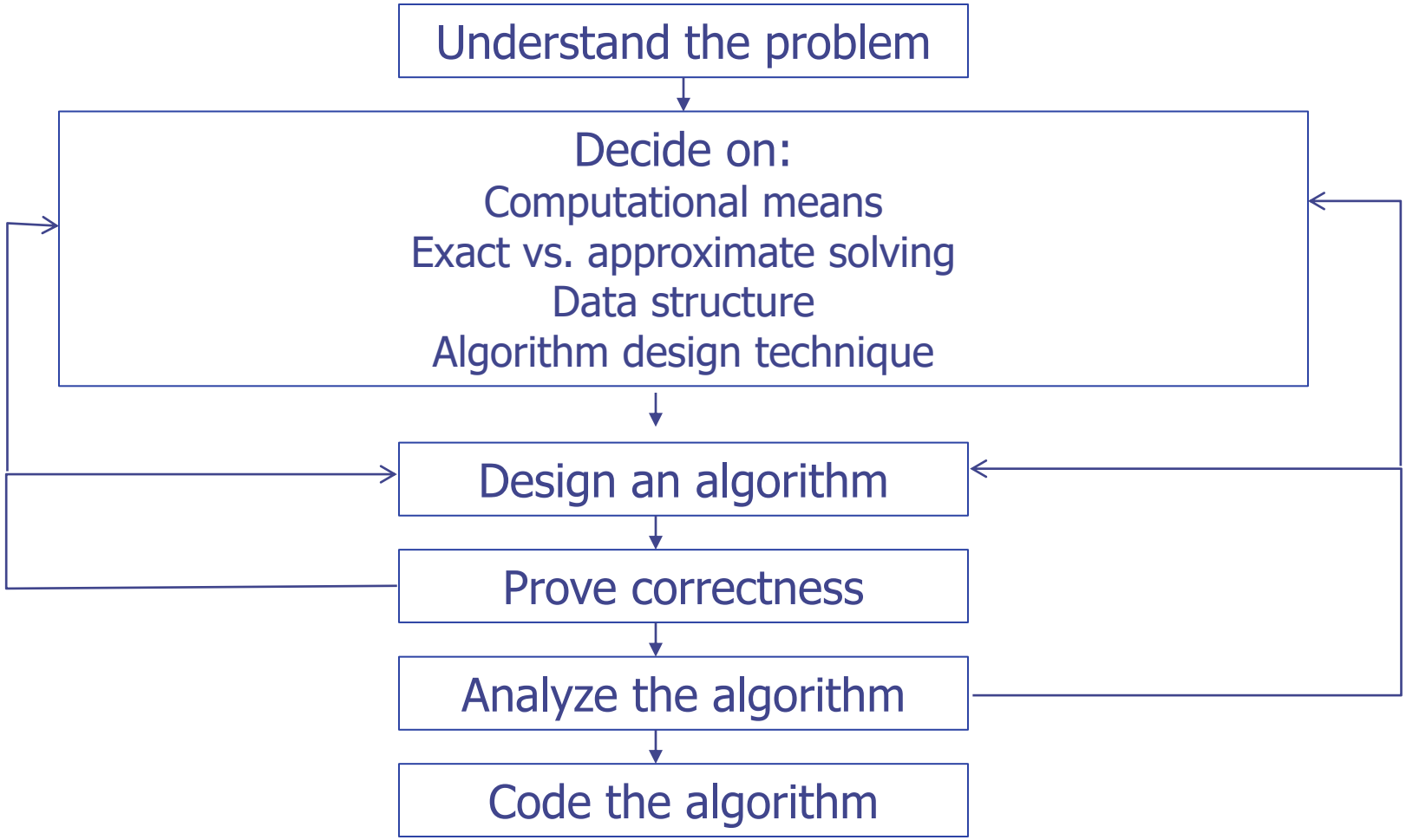
  Sort a sequence of numbers into non-decreasing order

  - Input: A sequence of n numbers (a_1, a_2, ….. a_n)
  - Output: A permutation (reordering) of the input sequence

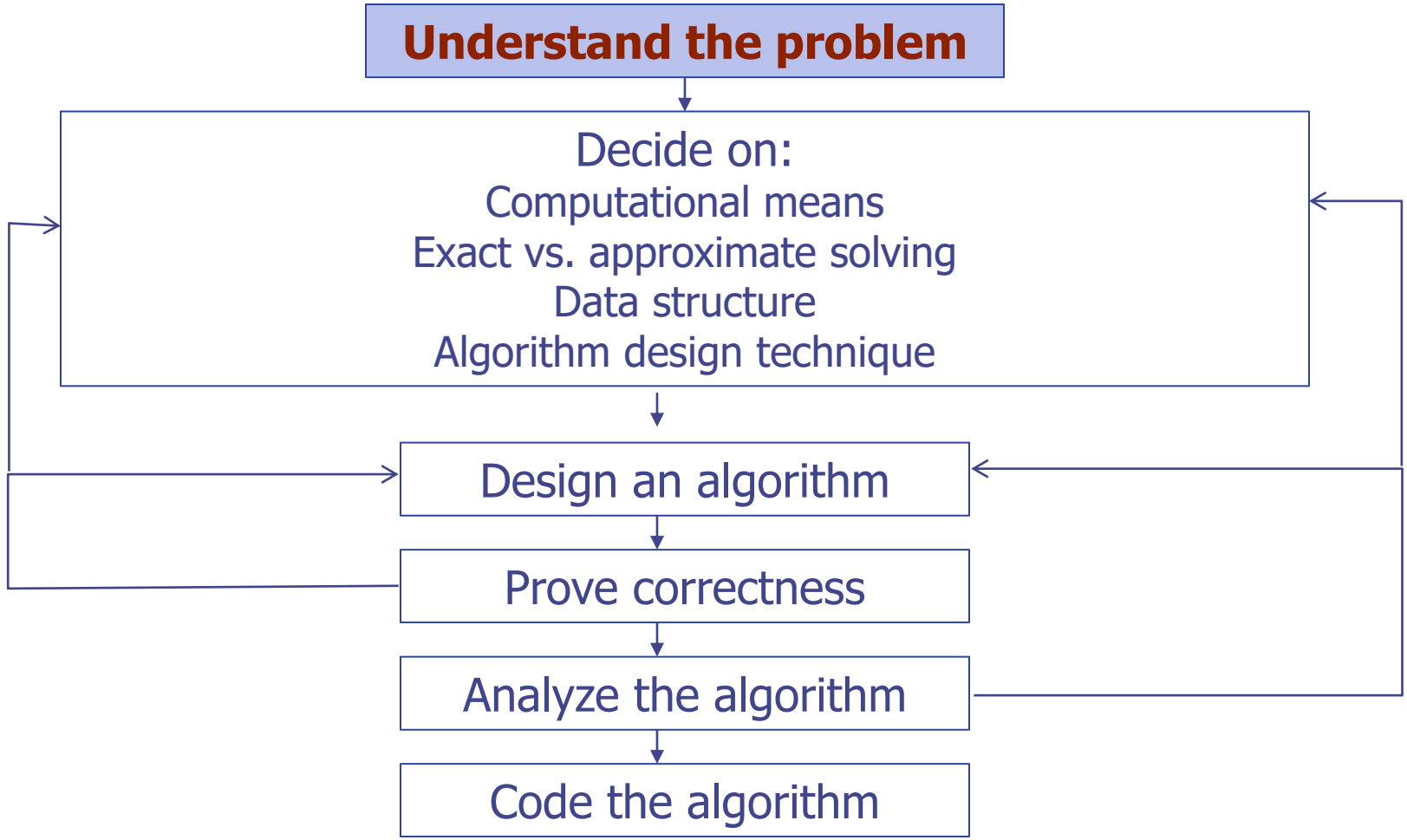
What is a Program?

- A program is the expression of an algorithm in a programming language.
- A set of concrete instructions which the computer will follow to solve a problem.

Algorithm Design and Analysis Process



Algorithm Design and Analysis Process



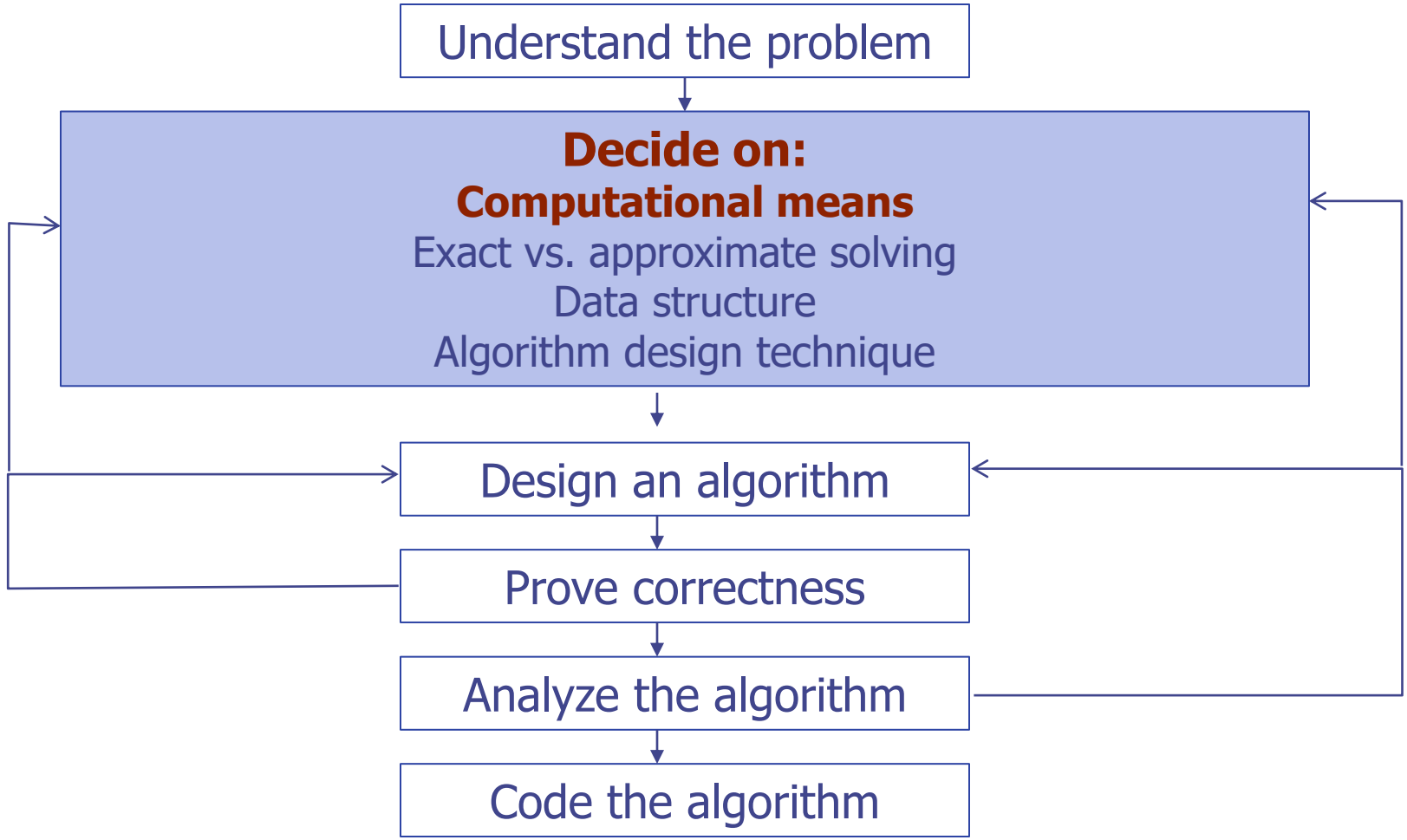
Understand the Problem

- Read the problem's description carefully and ask questions if you have any doubts
- Do a few small examples by hand
- Think about special cases, and ask questions again if needed.
- Try to find a similar reoccurring problem: sort, search, merge…
  - You might find a known algorithm

Known Problem Types

- Sorting: rearranging items of a given list.
- Searching: finding a key in a given list
- String processing: e.g. string matching
- Graph problems: e.g. TSP (Travelling Salesman Problem) and graph coloring problem.
- Combinatorial problems: ask for a combinatorial object that satisfies certain constraints and has some desired properties, such as permutation, and combination.
- Geometric problems: deal with geometric objects.
- Numerical problems: involve mathematical objects.

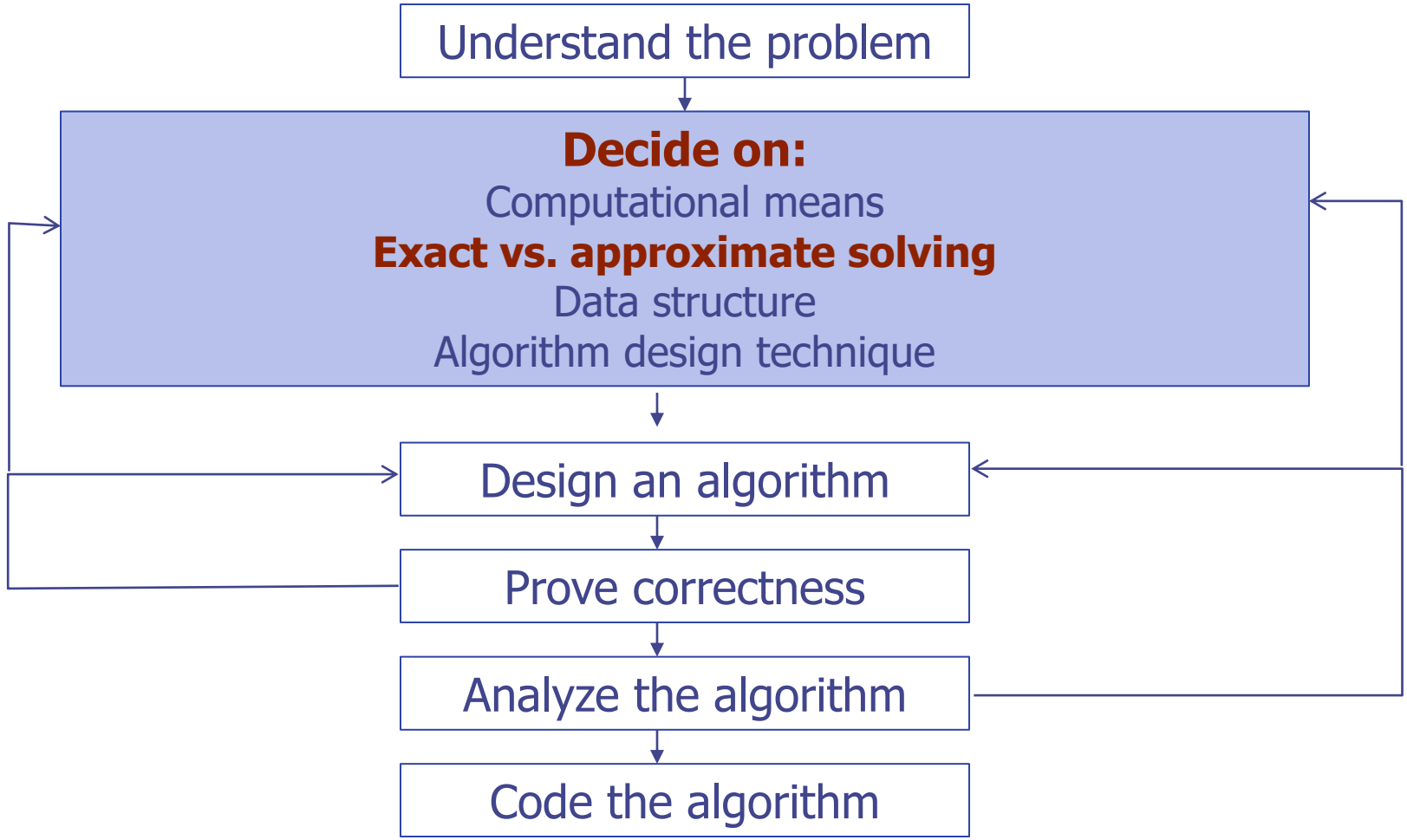
Algorithm Design and Analysis Process



Decide on: Computational Means

- Parallel or serial:
  - Todays' algorithms are still destined for RAM (random-access machine).i.e. one instruction at a time→ sequential algorithms
  - Newer computers can execute operations concurrently → parallel algorithms
- Speed and memory: no need to worry about, except for certain types of problems

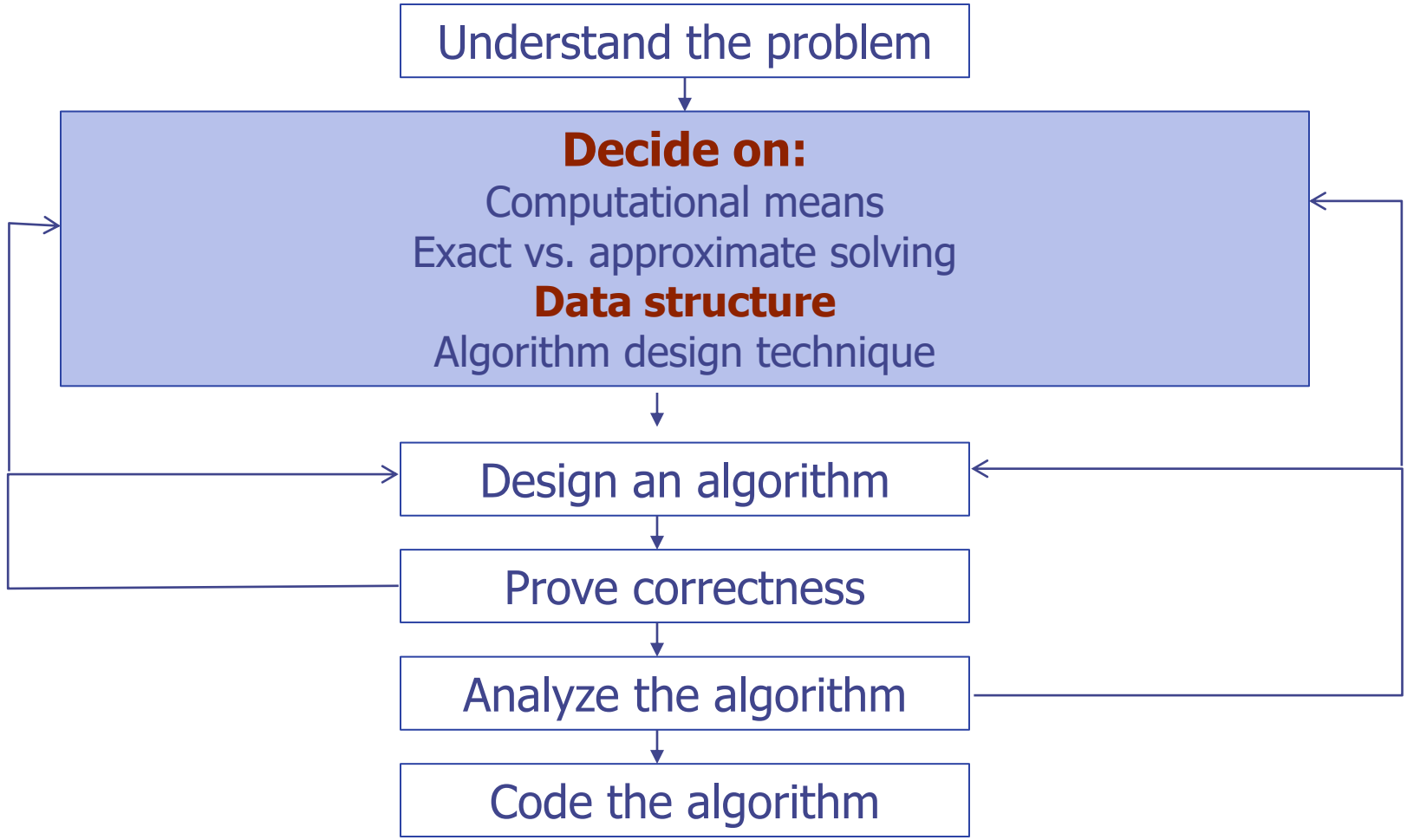
Algorithm Design and Analysis Process



Decide on: Exact vs. Approximate Solving

- Some important problems cannot be solved exactly for most of their instances.
  - E.g. extracting square roots, solving nonlinear equations, and evaluating definite integrals.
- Available algorithms for solving a problem exactly can be unacceptably slow because of the problem's intrinsic complexity.
  - Traveling Salesman Problem (TSP)
  - Knapsack Problem (KP)
- Approximation algorithm can be a part of a more sophisticated algorithm that solves a problem exactly

Algorithm Design and Analysis Process

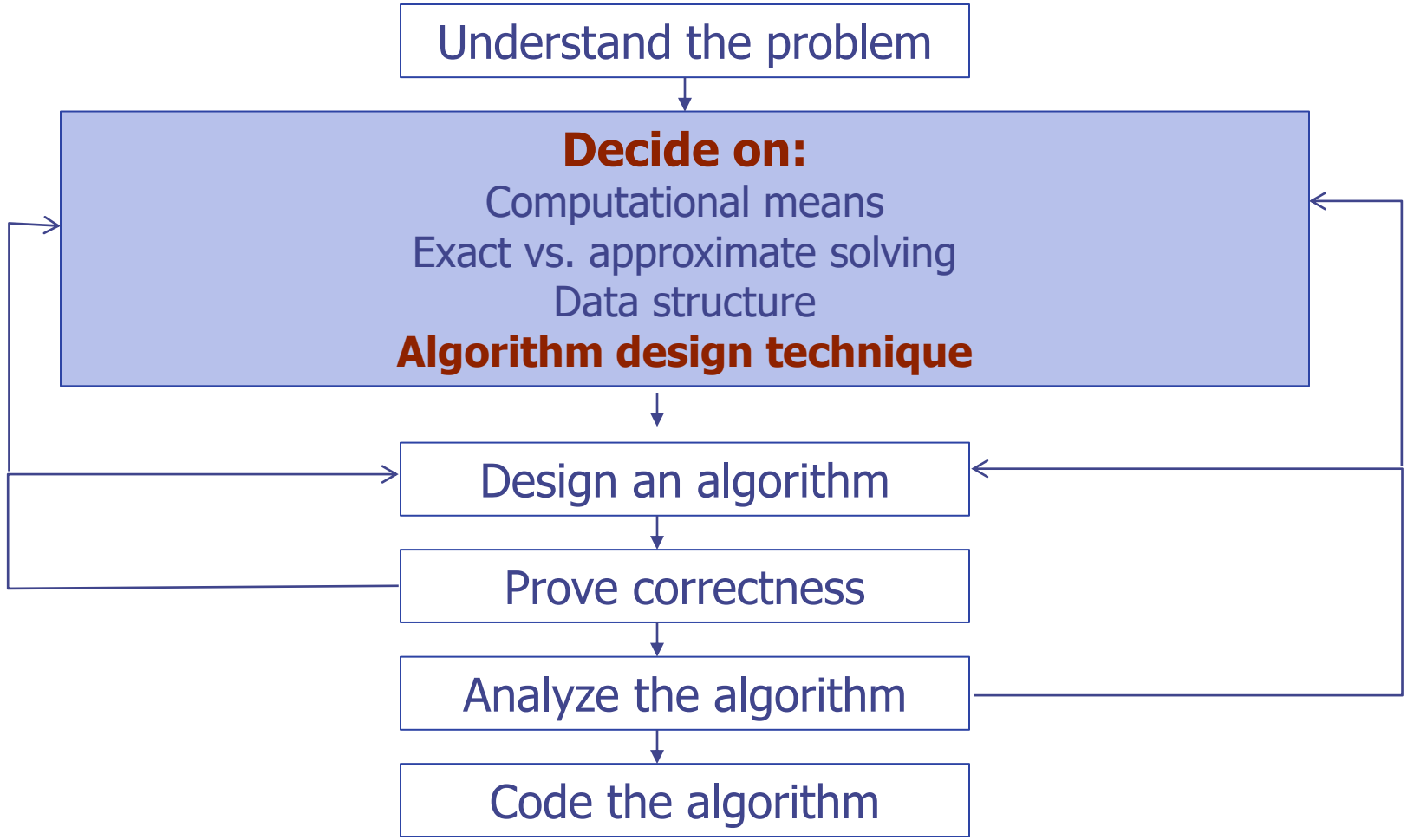


Decide on: Data Structures

- Cast your application in terms of well-studied abstract data structures

Concrete	***Abstract***
arrangement, ordering, sequence	permutation
cluster, collection, committee, group, packaging, selection	subsets
hierarchy, ancestor/descendants, taxonomy	trees
network, circuit, web, relationship	graph
sites, positions, locations	points
shapes, regions, boundaries	polygons
text, characters, patterns	strings

Algorithm Design and Analysis Process

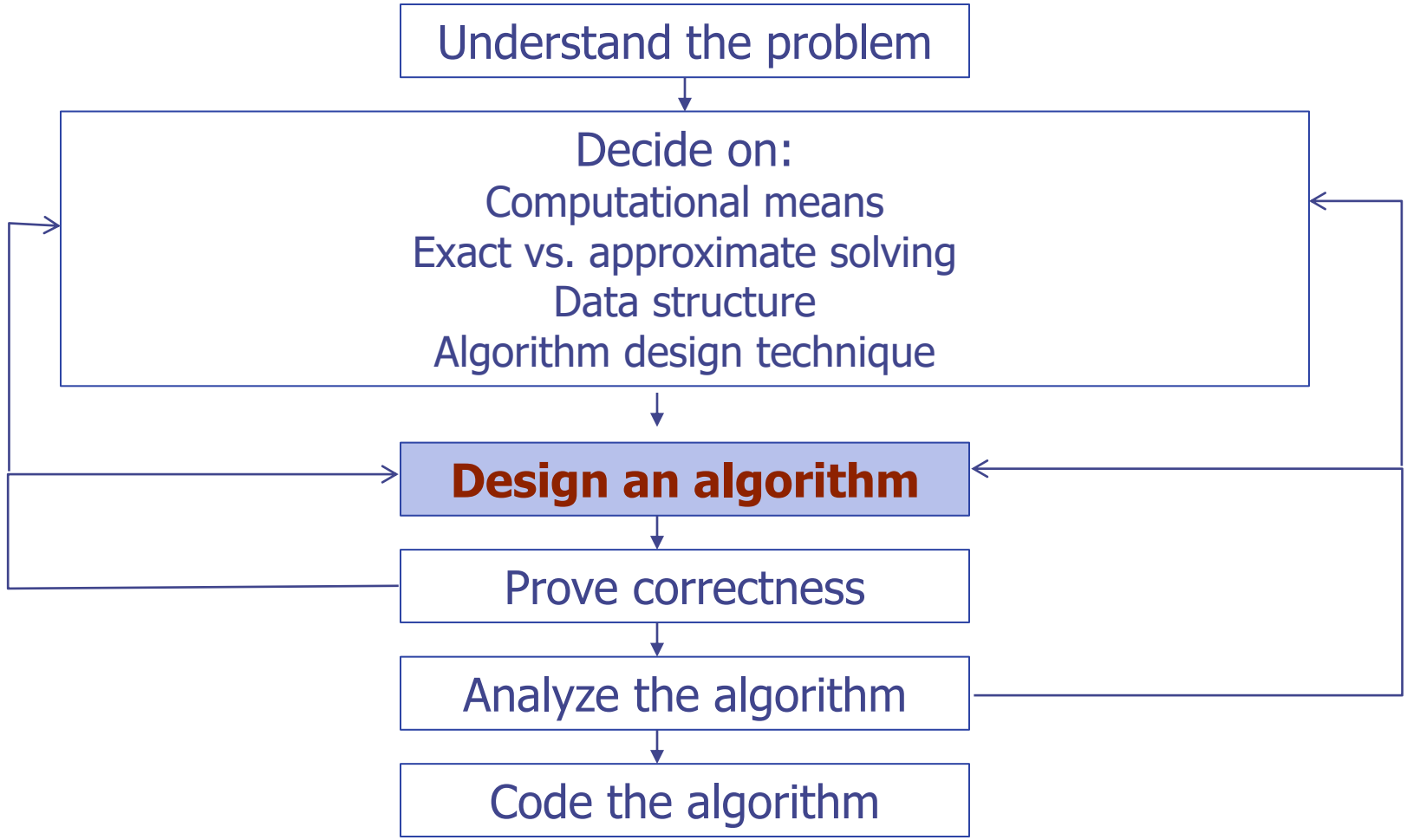


Decide on: Algorithm Design Technique

General approaches to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

- Brute force
- Divide and conquer
- Decrease and conquer
- Transform and conquer
- Branch and bound
- Greedy approach
- …

Algorithm Design and Analysis Process



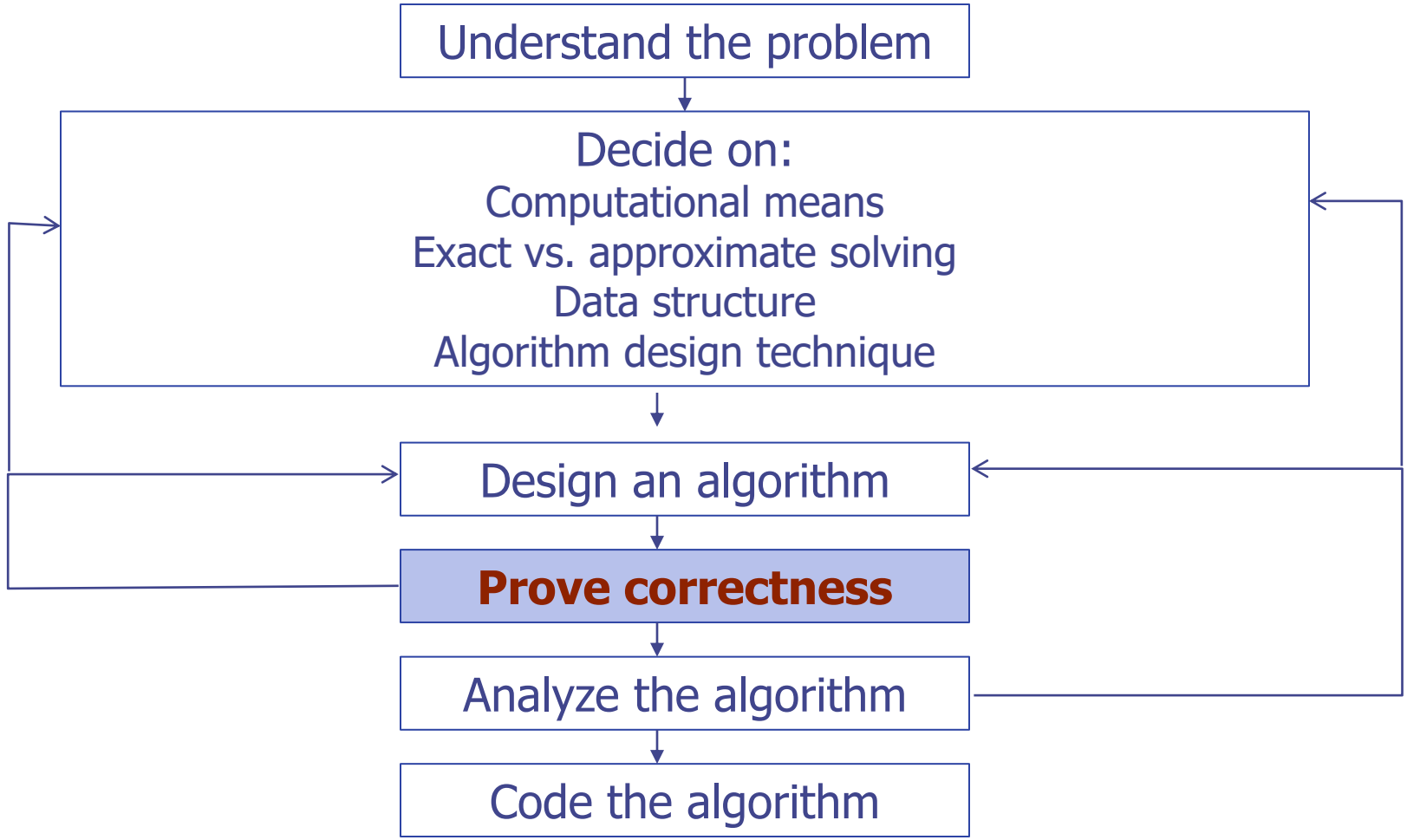
Design an algorithm -1

- Choose the suitable technique and apply it which is not an easy job because:
  - Some design techniques can be inapplicable for some problems
  - Sometimes, several techniques need to be combined
  - Some situations may need designing an algorithm for a particular problem which is a challenging task.

Design an algorithm -2

- Choose a suitable method to specify the algorithm:
  - Natural language: Adv. easy. Dis. ambiguous
  - Pseudocode: mixture of a natural language and programming language. Adv. more precise than NL. Dis. no single form.
  - Flowchart: old, suitable for simple algorithms only
- Express each step according to the design technique using the method you have chosen

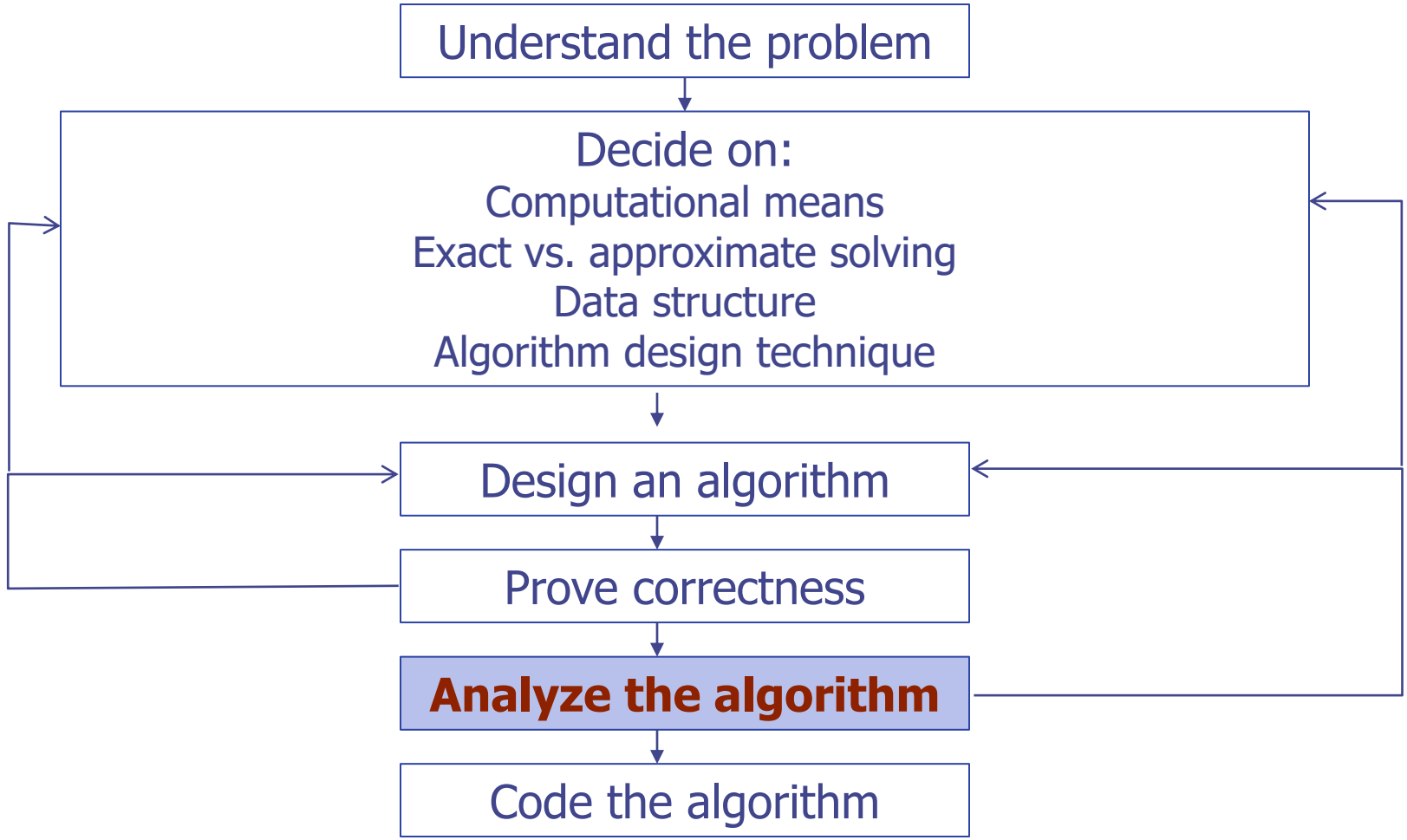
Algorithm Design and Analysis Process



Prove Correctness

- Does the algorithm yield the required result for every legitimate input in a finite amount of time?
- Mathematical induction is a common technique to prove correctness.
- Tracing the algorithm performance for specific inputs is useful but cannot prove correctness.
- To show that an algorithm is incorrect, you need just one instance of its input for which the algorithm fails

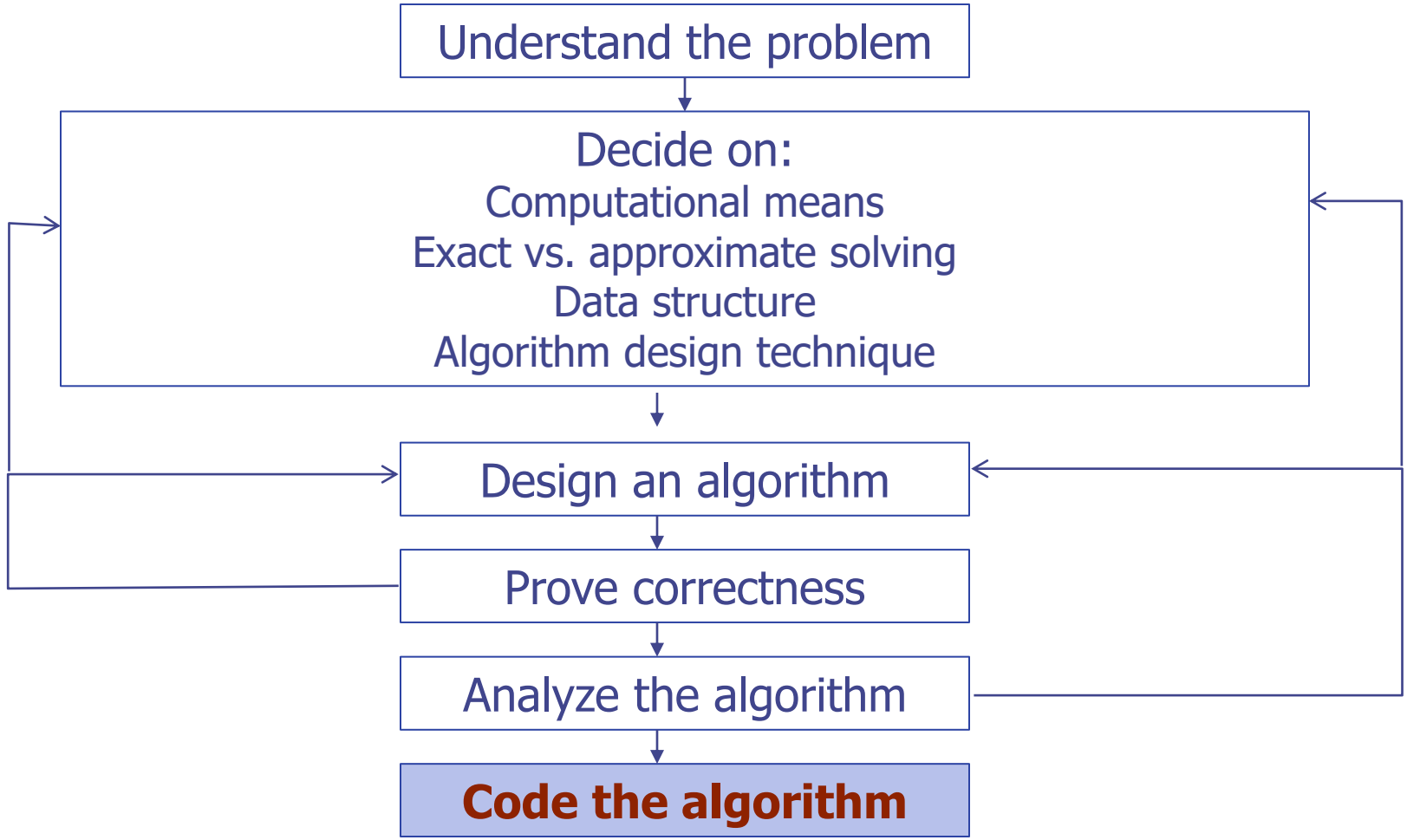
Algorithm Design and Analysis Process



Analyse the Algorithm

- How good is the algorithm?
  - Correctness
  - Time efficiency
  - Space efficiency
  - Simplicity.
- Does there exist a better algorithm?
  - Lower bounds
  - Optimality

Algorithm Design and Analysis Process



Code the Algorithm

- Choose a suitable programming language
- Implement the algorithm
- Test
- Debug
- Maintain

Inventing (or discovering?) algorithms is a very creative and rewarding process.

